

Paimon CodeGen

模块架构设计文档

核心架构设计 · 设计模式 · 流程图 · 时序图 · 调用链

模块：paimon-codegen

生成工具：AI Architecture Analyzer

生成日期：2026年03月27日

第1章 模块概述

1.1 模块定位

paimon-codegen 是 Apache Paimon 的运行时代码生成模块，负责在运行时动态生成高性能的 Java 类，包括 Projection（列投影）、RecordComparator（记录比较器）、NormalizedKeyComputer（归一化键计算器）和 RecordEqualiser（记录等值比较器）。该模块使用 Scala 字符串模板拼接技术生成 Java 源代码字符串，然后通过 paimon-common 模块中的编译工具（CompileUtils）将其编译为可执行类。

该模块的设计灵感源自 Apache Flink 的代码生成框架，通过在运行时生成特化代码（而非使用通用的解释执行），避免了虚方法调用和类型检查的开销，从而实现了高性能的数据操作。这是典型的运行时代码生成+动态编译架构。

1.2 核心目标

- 高性能数据操作 — 通过运行时代码生成消除虚方法调用和类型检查开销
- 全类型支持 — 覆盖 Paimon 所有数据类型（BOOLEAN/INT/LONG/FLOAT/DOUBLE/DECIMAL/STRING/BINARY/ARRAY/VECTOR/MAP/MULTISET/ROW/VARIANT/BLOB 等）
- 可扩展的代码生成体系 — 基于 Scala 模板的灵活代码生成，支持 4 种核心操作的特化生成
- SPI 插件化 — 通过 Java SPI 机制将实现与接口完全解耦

1.3 技术栈

技术	版本	用途
Java	8+	主入口类（CodeGeneratorImpl）和数据模型（SortSpec）
Scala	2.12.15	核心代码生成器的实现语言
paimon-common	同版本	提供 CodeGenerator 接口、数据类型系统、编译工具
maven-shade-plugin	-	将 Scala 依赖 shading 进 jar

1.4 源码文件一览

文件名	语言	行数	职责描述
CodeGeneratorImpl.java	Java	70	CodeGenerator 接口的唯一 SPI 实现类，模块入口
SortSpec.java	Java	157	排序规范描述类，定义排序字段、升降序、null 策略
CodeGenException.scala	Scala	23	代码生成统一异常类

文件名	语言	行数	职责描述
CodeGeneratorContext.scala	Scala	217	代码生成核心上下文，管理可复用代码片段和引用
GeneratedExpression.scala	Scala	47	表达式代码的不可变数据结构（case class）
GenerateUtils.scala	Scala	855	核心工具类，提供类型映射、字段读写、比较等生成能力
ExprCodeGenerator.scala	Scala	167	表达式代码生成器，组合多表达式为结果行
ProjectionCodeGenerator.scala	Scala	141	投影代码生成器，从行中选取指定列
ComparatorCodeGenerator.scala	Scala	75	比较器代码生成器，生成 RecordComparator
EqualiserCodeGenerator.scala	Scala	157	等值比较器代码生成器，生成 RecordEqualiser
ScalarOperatorGens.scala	Scala	366	标量运算符代码生成，支持各类等值/比较运算
SortCodeGenerator.scala	Scala	440	排序代码生成器，生成 NormalizedKeyComputer

第2章 核心架构设计

2.1 整体分层架构

paimon-codegen 采用经典的三层架构设计：接口层（paimon-common 定义）、入口层（Java SPI 实现）和生成器层（Scala 代码生成器）。接口层定义在 paimon-common 模块中，包括 CodeGenerator、Projection、RecordComparator 等核心接口。入口层通过 CodeGeneratorImpl 实现 SPI 机制，将请求委托给 Scala 生成器层。生成器层使用 Scala 字符串插值能力生成 Java 源代码。

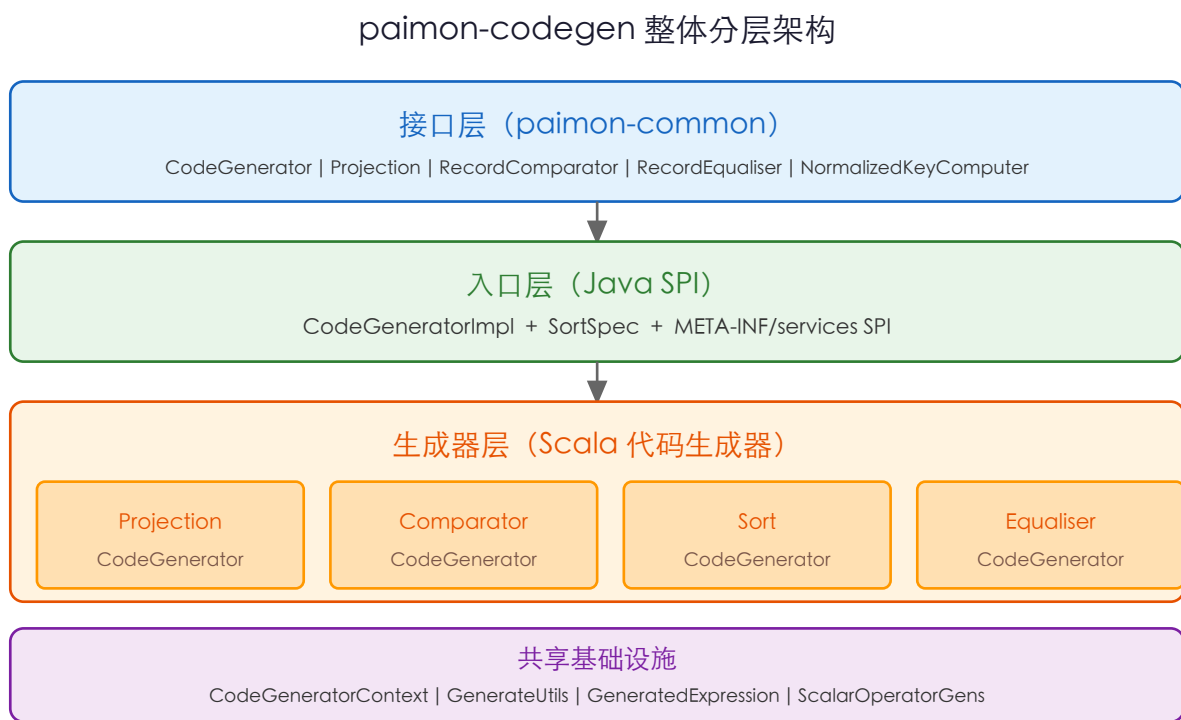


图2-1 paimon-codegen 整体分层架构图

2.2 类关系架构图

模块内部的类关系以委托模式为核心：CodeGeneratorImpl 作为统一入口，将四种代码生成请求分别委托给 ProjectionCodeGenerator、ComparatorCodeGenerator、SortCodeGenerator 和 EqualiserCodeGenerator。所有生成器共享 CodeGeneratorContext（上下文）和 GenerateUtils（工具类）基础设施。

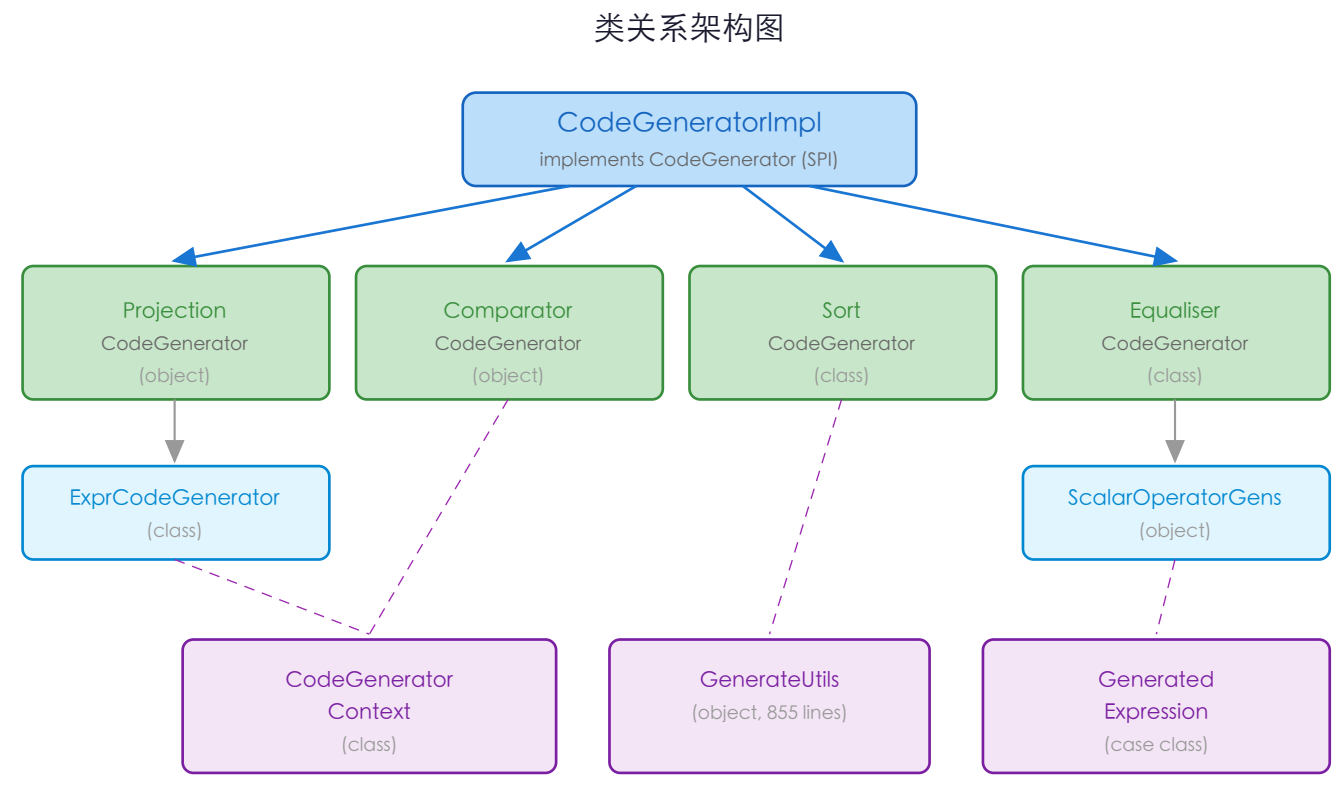


图2-2 paimon-codegen 类关系架构图

2.3 外部依赖关系

外部类/接口	所在模块	用途
CodeGenerator	paimon-common	代码生成器接口，CodeGeneratorImpl 实现它
GeneratedClass<T>	paimon-common	封装生成的代码，负责编译和实例化
Projection	paimon-common	投影接口 (BinaryRow apply(InternalRow))
RecordComparator	paimon-common	记录比较器接口
RecordEqualiser	paimon-common	记录等值比较器接口
NormalizedKeyComputer	paimon-common	归一化键计算器接口
DataType / RowType / ArrayType 等	paimon-common	完整的数据类型系统
InternalRow / BinaryRow 等	paimon-common	内部数据表示
CompileUtils / JavaCodeSplitter	paimon-common	运行时 Java 代码编译和代码拆分
InternalSerializers	paimon-common	类型序列化器工厂

第3章 核心类设计模式详解

3.1 SPI 服务提供者模式 — CodeGeneratorImpl

模式说明：CodeGeneratorImpl 通过 Java SPI (Service Provider Interface) 机制注册为 CodeGenerator 接口的唯一实现。SPI 配置文件位于 META-INF/services/org.apache.paimon.codegen.CodeGenerator，内容为 org.apache.paimon.codegen.CodeGeneratorImpl。这使得 paimon-common 模块无需直接依赖 paimon-codegen 即可通过 SPI 发现并加载代码生成器。

关键方法：

- generateProjection(RowType, int[]) — 委托给 ProjectionCodeGenerator 生成列投影代码
- generateNormalizedKeyComputer(List, int[]) — 委托给 SortCodeGenerator 生成归一化键计算代码
- generateRecordComparator(List, int[], boolean) — 委托给 ComparatorCodeGenerator 生成记录比较代码
- generateRecordEqualiser(List, int[]) — 委托给 EqualiserCodeGenerator 生成等值比较代码

3.2 外观模式 (Facade) — CodeGeneratorImpl

模式说明：CodeGeneratorImpl 同时扮演外观 (Facade) 角色，为外部调用者提供统一的 4 个方法入口，隐藏了内部复杂的 Scala 生成器实现、上下文管理和代码拼接细节。外部只需传入类型和字段索引即可获得 GeneratedClass 对象。

3.3 Builder 模式 — SortSpec.SortSpecBuilder

模式说明：SortSpec 使用经典的 Builder 模式构建排序规范。SortSpecBuilder 提供 addField(fieldIndex, isAscendingOrder, nullsLast) 方法逐步添加排序字段，最终通过 build() 生成不可变的 SortSpec 对象。这使排序规范的构建过程清晰、灵活。

关键方法：

- SortSpec.builder() — 创建 SortSpecBuilder 实例
- addField(int, boolean, boolean) — 添加排序字段
- build() — 构建不可变 SortSpec 对象

3.4 上下文对象模式 — CodeGeneratorContext

模式说明：CodeGeneratorContext

作为所有代码生成器共享的上下文对象，统一管理代码生成过程中的各类可复用状态，包括 references (引用对象列表)、reusableMemberStatements (成员变量声明)、reusableInitStatements (构造函数语句)、reusableLocalVariableStatements (局部变量声明，按方法名分组) 和

reusableTypeSerializers（类型序列化器缓存）。

关键方法：

- addReusableLocalVariable(typeTerm, name) — 添加可复用局部变量
- addReusableObject(obj, prefix, typeTerm) — 添加引用对象到成员区域
- addReusableTypeSerializer(DataType) — 添加类型序列化器（带缓存）
- reuseMemberCode() / reuseInitCode() / reuseLocalVariableCode() — 输出收集的代码片段

3.5 策略模式 — GenerateUtils 类型分派

模式说明：GenerateUtils 中的核心方法（如 generateCompare、rowFieldReadAccess、binaryWriterWriteField 等）根据 DataType 的 TypeRoot

枚举值选择不同的代码生成策略。每种数据类型对应特定的字段读取方式和比较方式，通过 Scala 的模式匹配（pattern matching）实现策略分派，避免了大量 if-else 或 Strategy 接口。

3.6 值对象模式 — GeneratedExpression

模式说明：GeneratedExpression 是 Scala case class，作为不可变的值对象承载生成表达式的四个组成部分：resultTerm（结果变量名）、nullTerm（null 标识变量名）、code（生成代码）、resultType（结果类型）。伴生对象定义了常量 ALWAYS_NULL、NEVER_NULL、NO_CODE。

3.7 Singleton 模式 — Scala Object

模式说明：GenerateUtils、ComparatorCodeGenerator、ProjectionCodeGenerator、Scalar OperatorGens 均为 Scala object 单例，天然实现了 Singleton 模式，保证全局唯一实例，适合无状态的工具类/工厂。

3.8 模板方法模式 — 代码模板拼接

模式说明：所有 Scala 代码生成器使用 Scala 字符串插值（s"... " 和 stripMargin）作为代码模板，将变量名、类型名、生成的代码片段嵌入固定的 Java 类模板中。这实质上是模板方法模式的变体，模板结构（类声明、构造函数、方法签名）固定，具体内容（字段读取、比较逻辑）由各生成器填充。

第4章 核心流程图

4.1 Projection 代码生成流程

Projection 代码生成是最常用的操作之一，从输入 `InternalRow` 中选取指定列生成新的 `BinaryRow`。流程涉及 `CodeGeneratorImpl`、`ProjectionCodeGenerator`、`ExprCodeGenerator`、`GenerateUtils` 和 `CodeGeneratorContext` 的协作。

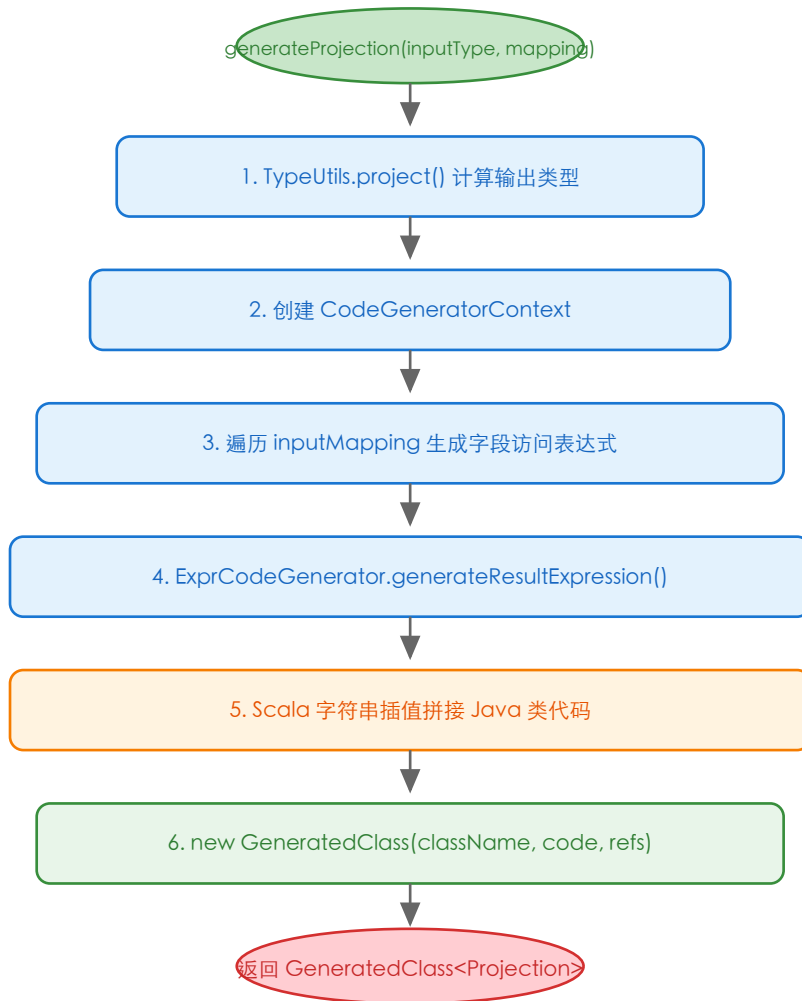


图4-1 Projection 代码生成流程图

4.2 NormalizedKey 排序代码生成流程

`SortCodeGenerator` 负责生成 `NormalizedKeyComputer` 实现类，该流程涉及归一化键长度计算、chunk 分块策略、大小端字节序处理等复杂逻辑。

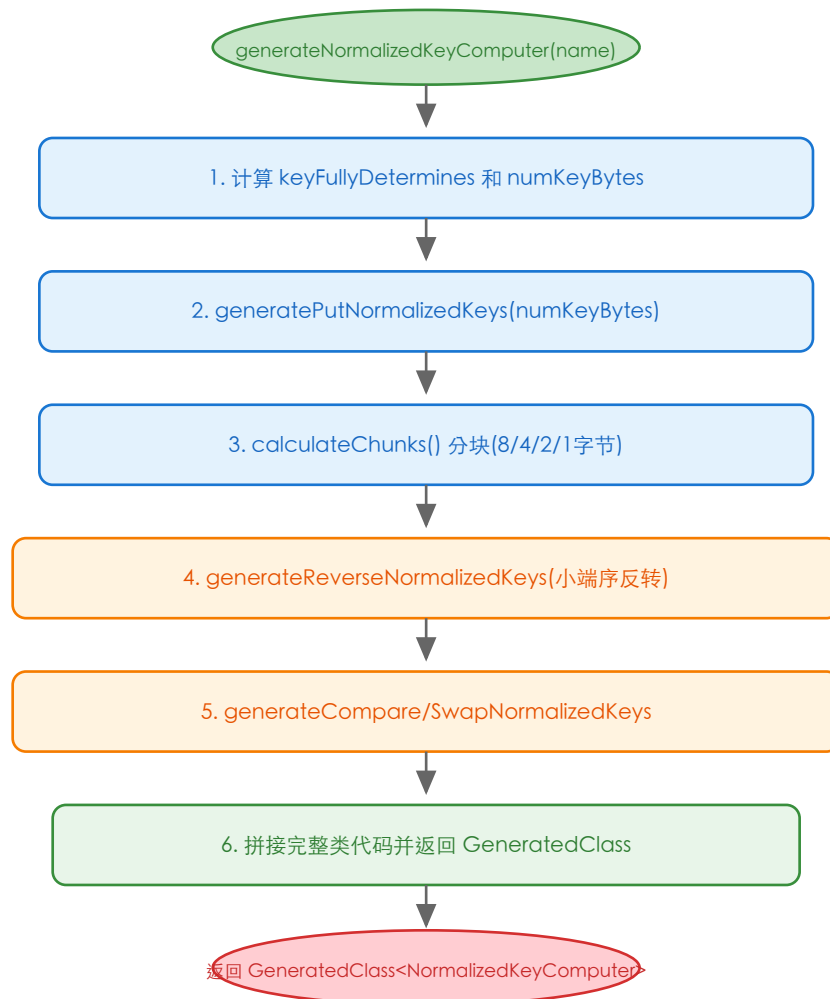


图4-2 NormalizedKeyComputer 排序代码生成流程图

4.3 RecordEqualiser 等值比较代码生成流程

EqualiserCodeGenerator

为每个字段生成独立的

equalsAtIndex\$idx

方法，支持字段投影、BinaryRow 快速路径优化、基本类型/复合类型/集合类型的差异化比较。

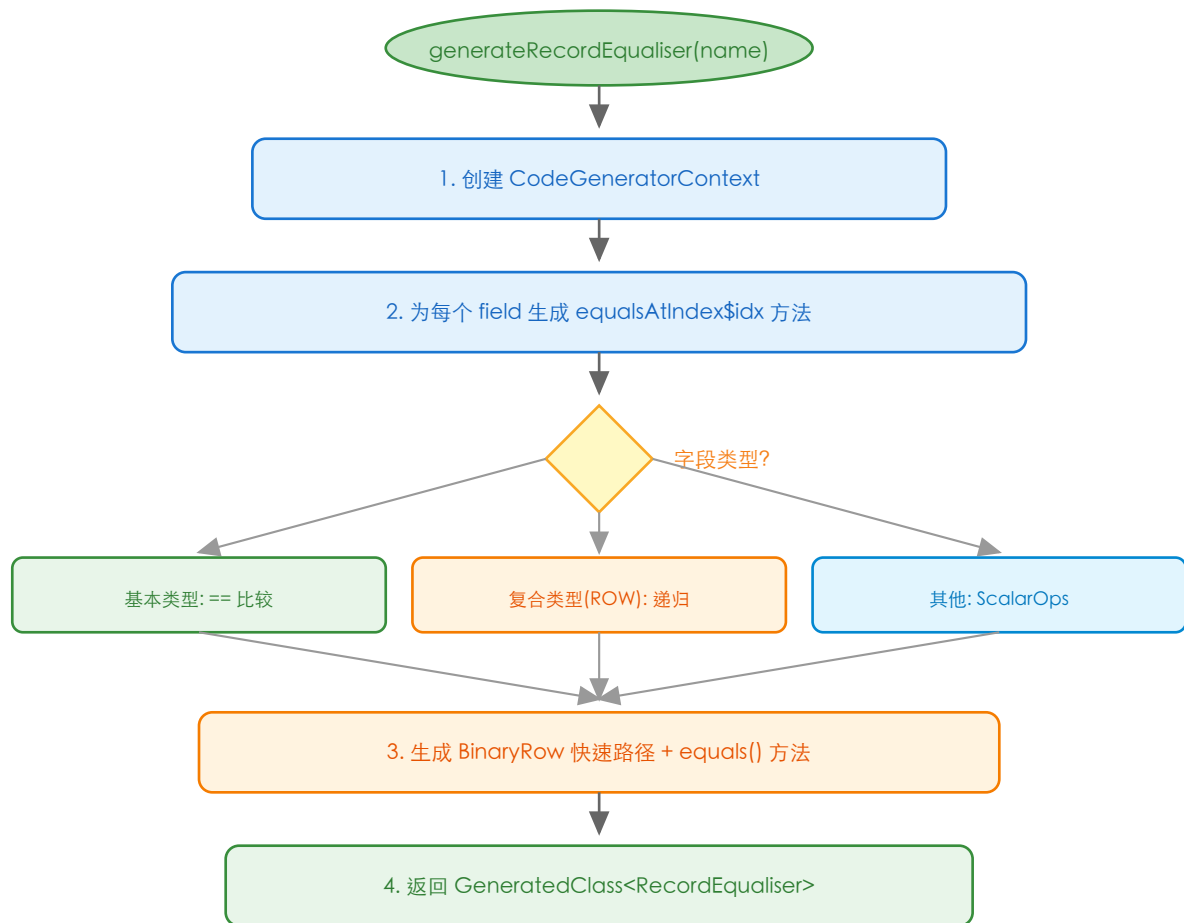


图4-3 RecordEqualiser 等值比较代码生成流程图

第5章 核心时序图

5.1 Projection 生成与实例化时序图

该时序图展示了从用户调用 `CodeGenerator.generateProjection()` 到获得可执行 `Projection` 实例的完整交互过程。

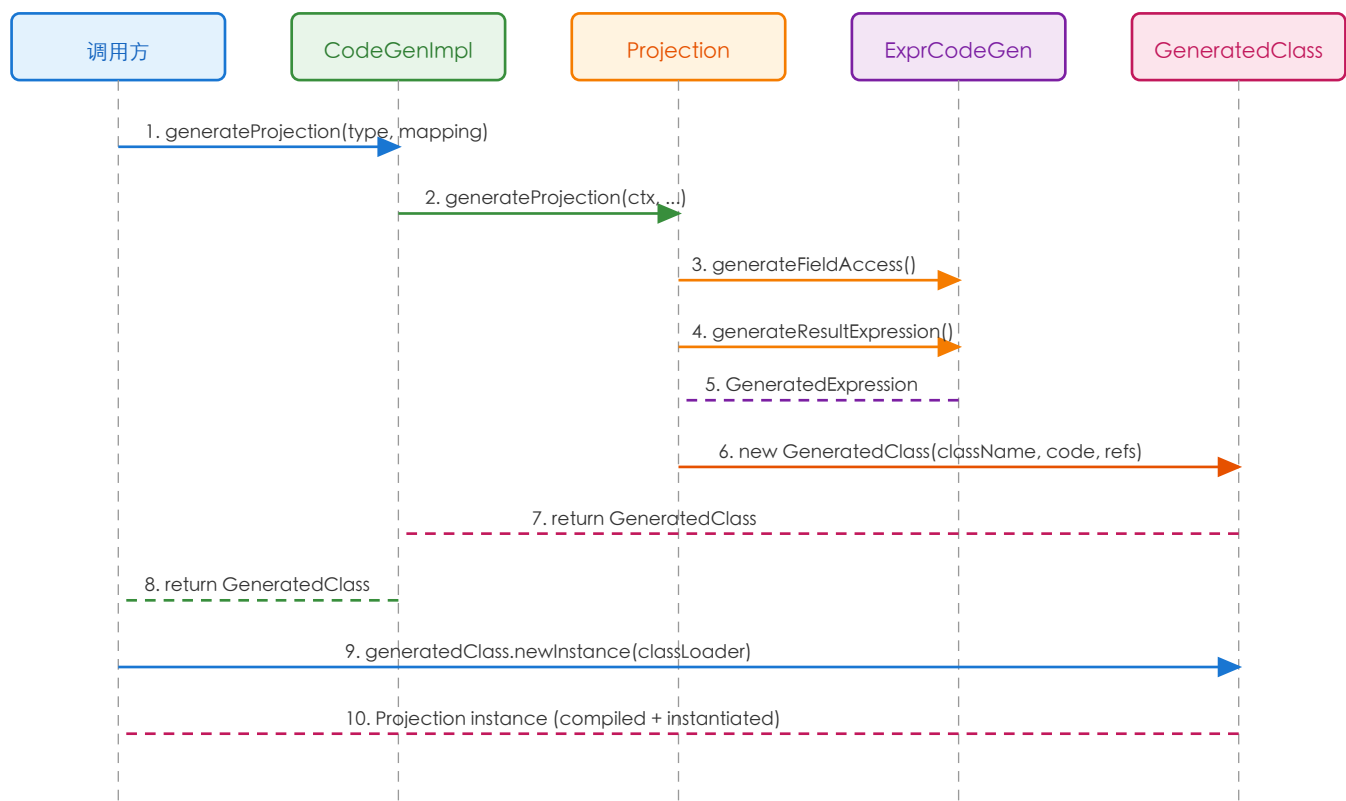


图5-1 Projection 生成与实例化时序图

5.2 RecordComparator 生成时序图

展示 `RecordComparator` 代码生成过程中各组件的交互。

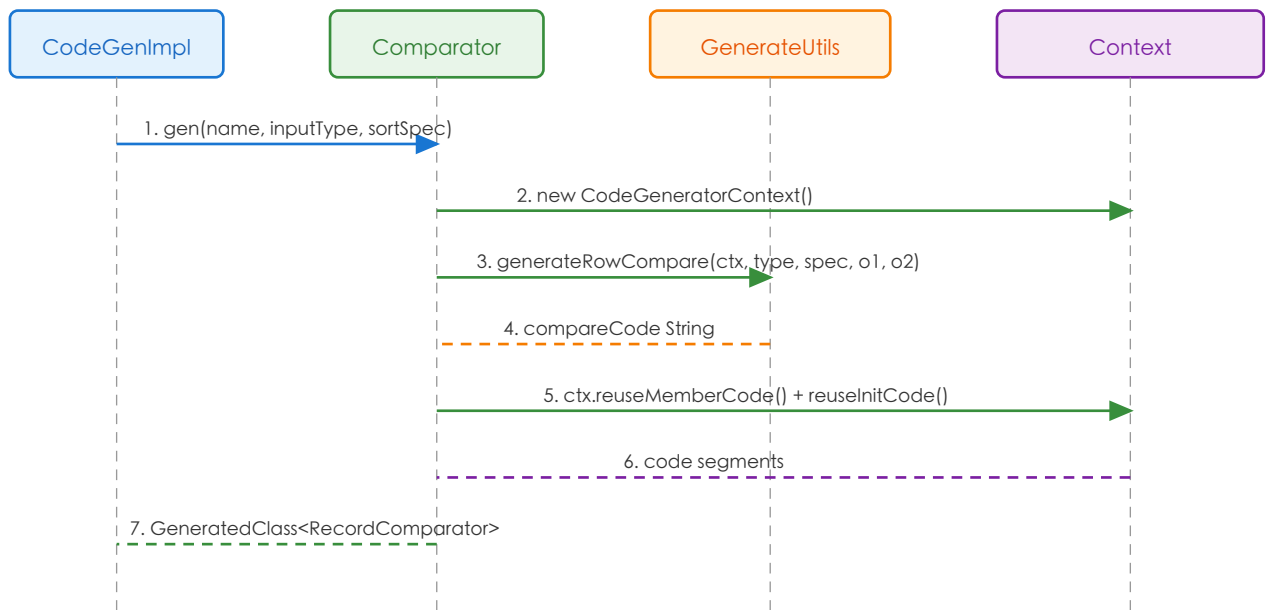


图5-2 RecordComparator 生成时序图

5.3 EqualiserCodeGenerator 内部交互时序图

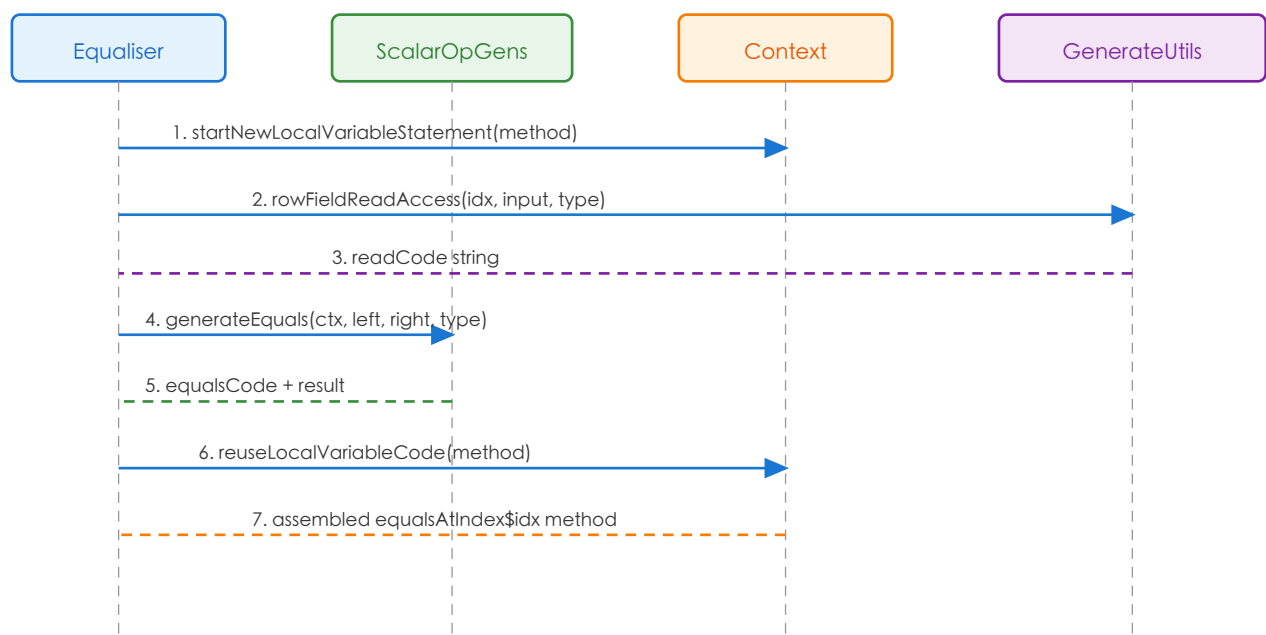


图5-3 EqualiserCodeGenerator 字段等值比较生成时序图

第6章 关键场景调用链

6.1 场景一：生成列投影 Projection

触发条件：需要从行数据中选取指定列时（如 Schema Evolution、列裁剪）

步骤	类.方法	说明
1	CodeGeneratorImpl.generateProjection(inputType, inputMapping)	入口，计算 <code>outputType = TypeUtils.project(inputType, mapping)</code>
2	ProjectionCodeGenerator.generateProjection(ctx, name, inType, outType, mapping)	创建 <code>ExprCodeGenerator</code> ，遍历 <code>mapping</code> 生成字段访问表达式
3	GenerateUtils.generateFieldAccess(ctx, inputType, inputTerm, idx)	为每个字段生成 <code>isNullAt()</code> + <code>getXxx()</code> 的访问代码
4	ExprCodeGenerator.generateResultExpression(fieldExprs, returnType, BinaryRow.class)	类型检查 + 字段设置代码 + <code>BinaryRowWriter reset/complete</code>
5	GenerateUtils.rowSetField(ctx, BinaryRow.class, outRow, pos, fieldExpr, writer)	生成 <code>writer.writeXxx()</code> 或 <code>writer.setNullAt()</code> 代码
6	<code>new GeneratedClass(className, code, ctx.references.toArray())</code>	封装生成的 Java 源代码，后续通过 <code>CompileUtils</code> 编译

6.2 场景二：生成记录比较器 RecordComparator

触发条件：数据排序时（如 MergeSort、排序合并）

步骤	类.方法	说明
1	CodeGeneratorImpl.generateRecordComparator(inputTypes, sortFields, isAsc)	入口，构建 <code>SortSpec</code>
2	SortSpec.SortSpecBuilder.addField(field, isAsc, false)	使用 <code>Builder</code> 逐步添加排序字段
3	ComparatorCodeGenerator.gen(name, inputType, sortSpec)	创建上下文，委托 <code>GenerateUtils</code>
4	GenerateUtils.generateRowCompare(ctx, inputType, sortSpec, o1, o2)	遍历排序字段，生成 <code>null</code> 检查 + 字段读取 + 比较代码

步骤	类.方法	说明
5	GenerateUtils.generateCompare(ctx, type, nullsIsLast, leftTerm, rightTerm)	按数据类型生成比较表达式 (compareTo/==/>/<)
6	new GeneratedClass(className, code, ctx.references.toArray)	封装为可编译的 GeneratedClass

6.3 场景三：生成归一化键计算器 NormalizedKeyComputer

触发条件：外部排序需要高效二进制键比较时

步骤	类.方法	说明
1	CodeGeneratorImpl.generateNormalizedKeyComputer(inputTypes, sortFields)	入口，创建 SortCodeGenerator
2	SortCodeGenerator 构造函数	计算 nullAwareNormalizedKeyLen、normalizedKeyNum、invertNormalizedKey
3	getKeyFullyDeterminesAndBytes()	判断归一化键是否完全确定排序，计算键字节数
4	generatePutNormalizedKeys(numKeyBytes)	为每个排序字段生成 SortUtil.putXxxNormalizedKey() 代码
5	calculateChunks(numKeyBytes)	将键拆分为 8/4/2/1 字节块（性能优化）
6	generateReverseNormalizedKeys(chunks)	小端系统上反转字节序（大端比较需要）
7	generateCompare/SwapNormalizedKeys(chunks)	生成基于 chunk 的无符号比较和交换代码

6.4 场景四：生成等值比较器 RecordEqualiser

触发条件：数据去重、Changelog 检测等需要判断两行是否相等时

步骤	类.方法	说明
1	CodeGeneratorImpl.generateRecordEqualiser(fieldTypes, fields)	入口，创建 EqualiserCodeGenerator
2	EqualiserCodeGenerator.generateRecordEqualiser(name)	遍历字段生成 equalsAtIndex\$idx 方法
3	generateEqualsMethod(ctx, idx)	生成字段读取 + null 检查 + 类型判断代码

步骤	类.方法	说明
4	generateEqualsCode(ctx, type, leftField, rightField, ...)	基本类型用 ==，复合类型递归生成 equaliser，其他委托 ScalarOperatorGens
5	ScalarOperatorGens.generateEquals(ctx, left, right, resultType)	支持 String/Numeric/Array/Vector/Map/Multiset/Composite 类型的等值比较
6	BinaryRow 快速路径	当两侧都是 BinaryRow 且无忽略字段时，直接 equals()

6.5 场景五：GeneratedClass 编译与实例化

触发条件：GeneratedClass.newInstance(classLoader) 被调用时

步骤	类.方法	说明
1	GeneratedClass.newInstance(classLoader)	外部调用入口
2	GeneratedClass.compile(classLoader)	首次调用时编译代码（结果被缓存）
3	JavaCodeSplitter.split(code, 4000, 10000)	拆分大方法避免 JVM 64KB 方法体限制
4	CompileUtils.compile(classLoader, className, splitCode)	运行时编译 Java 源代码为 Class 对象
5	clazz.getConstructor(Object[].class).newInstance(references)	通过反射调用构造函数，传入 references 数组
6	返回 Projection/RecordComparator 等实例	可直接用于数据操作

第7章 配置与扩展

7.1 SPI 扩展机制

paimon-codegen 通过 Java SPI 机制暴露 CodeGenerator 实现。SPI 配置文件位于：

```
META-INF/services/org.apache.paimon.codegen.CodeGenerator
// 自定义实现类：
org.apache.paimon.codegen.CodeGeneratorImpl
```

如需替换代码生成器实现，只需在 classpath 中提供另一个 SPI 配置文件，指向自定义的 CodeGenerator 实现类。

7.2 Maven 构建配置

模块使用 scala-maven-plugin 在 process-resources 阶段编译 Scala 代码（先于 Java 编译），并使用 maven-shade-plugin 将 Scala 标准库 shading 进最终 jar 以避免类路径冲突。

构建插件	阶段	用途
scala-maven-plugin	process-resources	编译 Scala 源代码（先于 Java）
scala-maven-plugin	process-test-resources	编译 Scala 测试代码
maven-shade-plugin	package	Shading Scala 依赖（minimizeJar=true）

7.3 内部扩展点

模块内部提供以下扩展点，支持新增数据类型和操作：

- GenerateUtils.rowFieldReadAccess() — 新增数据类型的字段读取代码生成
- GenerateUtils.generateCompare() — 新增数据类型的比较代码生成
- GenerateUtils.binaryWriterWriteField() — 新增数据类型的字段写入代码生成
- ScalarOperatorGens.generateEquals() — 新增数据类型的等值比较代码生成
- SortCodeGenerator.supportNormalizedKey() — 新增数据类型的归一化键支持

第8章 设计亮点与总结

8.1 设计亮点

- 运行时代码生成消除类型解释开销：通过在运行时为特定 Schema 生成特化的 Java 代码，避免了通用代码中的虚方法调用、类型检查和装箱/拆箱开销，显著提升数据密集型操作的性能。
- Scala 字符串模板 + Java 目标代码的混合策略：利用 Scala 强大的字符串插值能力（s"..." + stripMargin）生成 Java 源代码，兼具模板的灵活性和目标语言的性能优势。
- SPI 解耦设计：paimon-common 定义接口，paimon-codegen 通过 SPI 提供实现，使得核心模块无需直接依赖 Scala 运行时和代码生成逻辑。
- NormalizedKey Chunk 优化：SortCodeGenerator 将归一化键按 8/4/2/1 字节分块，生成利用原生类型（long/int/short/byte）直接比较的代码，避免逐字节比较的开销。
- BinaryRow 快速路径：EqualiserCodeGenerator 对两侧都是 BinaryRow 的情况直接调用 equals()，跳过字段级别比较，利用二进制数据的连续内存布局优势。
- 代码拆分容错：GeneratedClass 先尝试编译经 JavaCodeSplitter 拆分后的代码，失败时回退到原始代码编译，确保可靠性。

8.2 设计限制

- Scala 2.12 强依赖：模块核心使用 Scala 编写，引入了 Scala 编译器和运行时依赖，虽然通过 maven-shade-plugin shading 缓解了类路径冲突，但增加了包体积和构建复杂度。
- 调试困难：运行时生成的代码难以设置断点调试，出错时错误信息可能不够直观。
- 代码生成器维护成本：新增数据类型需要同时修改多个生成器（GenerateUtils、ScalarOperatorGens、EqualiserCodeGenerator 等），容易遗漏导致部分场景不支持。

8.3 质量评估

评估维度	评分	说明
代码结构	★★★★★	三层架构清晰，职责分明，SPI 解耦优秀
设计模式运用	★★★★★	SPI、Facade、Builder、Context Object、Strategy、Singleton、Template Method 等 8 种模式灵活运用
性能优化	★★★★★	NormalizedKey chunk 优化、BinaryRow 快速路径、类型特化代码生成
可扩展性	★★★★☆	新数据类型需修改多处，但扩展点明确
可维护性	★★★☆☆	Scala + Java 混合编程增加理解成本，运行时生成代码调试困难
测试覆盖	★★★★☆	EqualiserCodeGeneratorTest 覆盖全部数据类型，参数化测试优秀

8.4 总结

paimon-codegen 模块是 Apache Paimon 高性能数据操作的核心基础设施。通过运行时代码生成+动态编译的架构，实现了针对具体 Schema 的高度特化代码，消除了通用数据操作中的性能开销。模块设计精良，灵活运用了 SPI、Facade、Builder、Context Object、Strategy 等设计模式，整体代码量约 2700 行（含 Java 和 Scala），紧凑而功能完备。该设计方案源自 Apache Flink 代码生成框架，经过 Paimon 团队的精简和适配，在保持高性能的同时实现了更好的模块化和可维护性。